

Conditional Branched Autoregressive Q for Restaurant Actions

1 Setup

Let the structured restaurant action be

$$a = (t, x, y, z),$$

where

- $t \in \mathcal{T}(s)$ is the action type,
- $x \in \mathcal{X}(s, t)$ is the first argument,
- $y \in \mathcal{Y}(s, t, x)$ is the second argument,
- $z \in \mathcal{Z}(s, t, x, y)$ is the third argument.

For the current restaurant environment, the mapping is

$$t = \text{action_type}, \quad x = \text{object1}, \quad y = \text{location}, \quad z = \text{object2}.$$

The valid sets $\mathcal{T}(s)$, $\mathcal{X}(s, t)$, $\mathcal{Y}(s, t, x)$, and $\mathcal{Z}(s, t, x, y)$ are induced by the environment masks.

2 Conditional Branched Joint Q

We define a joint action-value function

$$Q(s, a) = Q(s, t, x, y, z),$$

and parameterize it autoregressively as

$$Q(s, t, x, y, z) = q_t(s, t) + q_x(s, t, x) + q_y(s, t, x, y) + q_z(s, t, x, y, z). \quad (1)$$

This is a *joint* Q-function because it scores the full structured action. It is also *autoregressive* because each later term is conditioned on earlier choices. The current implementation in this repository uses the dueling-style decomposition described in Section 6.

Interpretation.

- $q_t(s, t)$ is the stage contribution for the action type: a learned scalar function of the state and the chosen action type (a joint score over (s, t)).
- $q_x(s, t, x)$ is a learned, advantage-style contribution that jointly scores the state, action type, and first argument; it is a function of (s, t, x) and should be read as a value (score), not as a conditional probability over x given (s, t) .

- $q_y(s, t, x, y)$ is a learned contribution that jointly scores (s, t, x, y) and captures the incremental value of choosing y given the prefix.
- $q_z(s, t, x, y, z)$ is a learned contribution that jointly scores (s, t, x, y, z) and captures the incremental value of choosing z given the prefix.

Unlike an unconditional additive factorization, the argument terms here are joint score functions tied to the action prefix rather than statistical conditional distributions; they are not normalized over their arguments.

3 Suffix Value Functions

Define the optimal suffix values recursively:

$$V_x(s, t) = \max_{x \in \mathcal{X}(s, t)} [q_x(s, t, x) + V_y(s, t, x)], \quad (2)$$

$$V_y(s, t, x) = \max_{y \in \mathcal{Y}(s, t, x)} [q_y(s, t, x, y) + V_z(s, t, x, y)], \quad (3)$$

$$V_z(s, t, x, y) = \max_{z \in \mathcal{Z}(s, t, x, y)} q_z(s, t, x, y, z). \quad (4)$$

These are not separate environment values. They are the optimal continuation values induced by the autoregressive decomposition in Equation (1).

4 Greedy Decoding

The greedy action under this model is obtained by backward-consistent sequential maximization.

First choose the action type:

$$t^* = \arg \max_{t \in \mathcal{T}(s)} [q_t(s, t) + V_x(s, t)]. \quad (5)$$

Then choose the first argument:

$$x^* = \arg \max_{x \in \mathcal{X}(s, t^*)} [q_x(s, t^*, x) + V_y(s, t^*, x)]. \quad (6)$$

Then choose the second argument:

$$y^* = \arg \max_{y \in \mathcal{Y}(s, t^*, x^*)} [q_y(s, t^*, x^*, y) + V_z(s, t^*, x^*, y)]. \quad (7)$$

Finally choose the third argument:

$$z^* = \arg \max_{z \in \mathcal{Z}(s, t^*, x^*, y^*)} q_z(s, t^*, x^*, y^*, z). \quad (8)$$

This yields the greedy action

$$a^* = (t^*, x^*, y^*, z^*).$$

Key property. The decoder is now mathematically aligned with the model:

$$a^* \in \arg \max_a Q(s, a).$$

That alignment is the main reason to prefer this formulation over unconditional branch-wise argmax.

5 Bellman Target

For a transition (s, a, r, s', d) with terminal flag $d \in \{0, 1\}$, the standard Q-learning target is

$$\hat{Q}(s, a) = r + \gamma(1 - d) \max_{a'} Q_{\text{tgt}}(s', a'). \quad (9)$$

Under the autoregressive factorization, the greedy next action is

$$a'_* = (t'_*, x'_*, y'_*, z'_*),$$

where each component is chosen by the same suffix-maximizing decoder as in Equations (5)–(8), but using s' and the valid masks at s' .

Therefore

$$\max_{a'} Q_{\text{tgt}}(s', a') = Q_{\text{tgt}}(s', t'_*, x'_*, y'_*, z'_*). \quad (10)$$

For Double DQN, one would choose a'_* using the online network and evaluate it using the target network:

$$\hat{Q}(s, a) = r + \gamma(1 - d) Q_{\text{tgt}}(s', a'_*), \quad a'_* \in \arg \max_{a'} Q_{\text{online}}(s', a'). \quad (11)$$

6 Dueling-Style Conditional Architecture

A dueling-style decomposition can be added in a way that remains compatible with the autoregressive joint Q semantics. The clean version is to use a single state-value baseline

$$V(s),$$

followed by centered conditional advantage terms at each stage:

$$V(s) = f_{\text{value}}(h(s)), \quad (12)$$

$$A_t(s, t) = g_t(h(s), t), \quad (13)$$

$$A_x(s, t, x) = g_x(h(s), \phi_t(t), x), \quad (14)$$

$$A_y(s, t, x, y) = g_y(h(s), \phi_t(t), \phi_x(x), y), \quad (15)$$

$$A_z(s, t, x, y, z) = g_z(h(s), \phi_t(t), \phi_x(x), \phi_y(y), z). \quad (16)$$

The corresponding stage contributions are

$$q_t(s, t) = V(s) + (A_t(s, t) - \bar{A}_t(s)), \quad (17)$$

$$q_x(s, t, x) = A_x(s, t, x) - \bar{A}_x(s, t), \quad (18)$$

$$q_y(s, t, x, y) = A_y(s, t, x, y) - \bar{A}_y(s, t, x), \quad (19)$$

$$q_z(s, t, x, y, z) = A_z(s, t, x, y, z) - \bar{A}_z(s, t, x, y), \quad (20)$$

where the centered baselines are computed over the valid masked choices at each stage:

$$\bar{A}_t(s) = \frac{1}{|\mathcal{T}(s)|} \sum_{t' \in \mathcal{T}(s)} A_t(s, t'), \quad (21)$$

$$\bar{A}_x(s, t) = \frac{1}{|\mathcal{X}(s, t)|} \sum_{x' \in \mathcal{X}(s, t)} A_x(s, t, x'), \quad (22)$$

$$\bar{A}_y(s, t, x) = \frac{1}{|\mathcal{Y}(s, t, x)|} \sum_{y' \in \mathcal{Y}(s, t, x)} A_y(s, t, x, y'), \quad (23)$$

$$\bar{A}_z(s, t, x, y) = \frac{1}{|\mathcal{Z}(s, t, x, y)|} \sum_{z' \in \mathcal{Z}(s, t, x, y)} A_z(s, t, x, y, z'). \quad (24)$$

The full joint Q remains

$$Q(s, t, x, y, z) = q_t(s, t) + q_x(s, t, x) + q_y(s, t, x, y) + q_z(s, t, x, y, z).$$

Implementation. This formulation is implemented by the `RestaurantQNetwork` class in `anticipatory_rl/agent`. The components are:

- $h(s)$ from `encoder` (line 200).
- $V(s)$ from `value_head` (line 206).
- $A_t(s, t)$ from `action_type_adv_head` (line 207).
- A_x, A_y, A_z from `object1_adv_head`, `location_adv_head`, and `object2_adv_head` respectively (lines 213–227).
- The centered contributions in Equations (17) and (18) are computed by the `*_scores` methods (lines 232–284).

Why this is better than stage-specific value heads. It is tempting to introduce separate learned value terms such as $V_x(s, t)$ or $V_y(s, t, x)$ and then add them to the stage contributions. However, that creates an identifiability problem: if two learned terms have the same conditioning context and are simply added together, the model can shift mass between them without changing the total Q-value. Using a single global baseline $V(s)$ plus centered conditional advantages avoids that redundancy.

Interpretation. The terms q_t, q_x, q_y, q_z should be understood as centered incremental contributions to the total joint Q-value, not as independent Bellman Q-functions. The actual continuation value after fixing a prefix is still given by the suffix maximization defined earlier in Section 3.

This can improve optimization, but it does not change the main semantic requirement: later stages must be conditioned on earlier ones, and decoding must remain aligned with the joint autoregressive Q.

7 Restaurant-Specific Instantiation

For the current environment, a natural specialization is

$$Q(s, \text{type}, \text{obj1}, \text{loc}, \text{obj2}) = q_{\text{type}}(s, \text{type}) + q_{\text{obj1}}(s, \text{type}, \text{obj1}) + q_{\text{loc}}(s, \text{type}, \text{obj1}, \text{loc}) + q_{\text{obj2}}(s, \text{type}, \text{obj1}, \text{loc}). \quad (25)$$

For action types that do not use some arguments, the corresponding valid set is a singleton containing the “none” token, so the same equations still apply without case-splitting.

8 Why This Fixes the Prior Design

The prior unconditional factorization effectively assumed that argument scores were reusable across unrelated action types. In contrast, the conditional branched model ensures:

- object preferences are action-type dependent,
- location preferences are conditioned on the action type and earlier arguments,
- the greedy decoder is consistent with the modeled Q-function,
- the Bellman target is computed with respect to the same joint action semantics used at inference time.